

Konzeption und Evaluation eines domänenbasierten Explorers als VS- Code Extension für SAP CAP Anwendungen

Projektskizze Bachelorarbeit

aus dem Studiengang Wirtschaftsinformatik Software Engineering

an der Dualen Hochschule Baden-Württemberg Mannheim

von

Max Christian Meinel

23.02.2026

Matrikelnummer, Kurs:	7864687, WWI23SEB
Unternehmen:	SAP SE, 69190, Walldorf
Abteilung:	CAP Tools & MTX
Leiter des Studiengangs:	Prof. Dr. Henning Pagnia
Betreuer im Unternehmen:	Christian Fuchs
Betreuer an der DHBW:	Ulrich Wolf

Inhaltsverzeichnis

1 Wissenschaftliche Ziele, Inhalte und Methodik	2
1.1 Ziele	2
1.2 Inhalte	2
1.3 Methodik	3
2 Zeitplanung	4
2.1 Plan A	4
2.2 Plan B	5
3 Vorläufige Gliederung	6
4 Literaturübersicht	8
Literatur	a

1 Wissenschaftliche Ziele, Inhalte und Methodik

1.1 Ziele

Das Ziel dieser Arbeit ist es eine VS-Code Extension zu konstruieren, welche Entwickler, die das SAP Cloud Application Programming Model (CAP) verwenden, bei ihrer Entwicklung unterstützen kann. Der Fokus der Unterstützung des Entwicklers liegt auf der Übersicht Innerhalb des Projekts. CAP Projekte können sehr schnell große Repositories werden, mit hunderten Endpunkten. Durch die Separation of Concerns, ist die Servicelogik bei CAP innerhalb des gesamten Repositries verteilt. Obwohl dieser Aufbau sehr vorteilhaft ist, erschwert das vor allem bei großen Repositories.

Bei diesem Problem soll die Erweiterung den Entwicklern eine Hilfestellung geben. Das konkrete Ziel ist es damit, dass die Erweiterung es ermöglicht alle domäinspezifische Logik, welche in den Ordnern verstreut ist zusammenzubringen und zu sortieren. Eine der Herausforderungen dabei wird es sein, dass es unterschiedliche Möglichkeiten der Sortierung und Filterung geben kann.

1.2 Inhalte

SAP CAP ist ein Framework, welches auf NodeJS aufbaut und für SAP optimiert ist. Es gibt einige Grundlagen von CAP, welche in der Arbeit erläutert werden müssen. Darunter fällt: CDS, CDL, CSN, LSP.

Bei dem Inhalt der Bachelorarbeit werden zwei Probleme fokussiert. Zum einem ist der Graphische Aufbau dieser Arbeit im Fokus. Dabei geht es nicht um die Ästhetik der Erweiterung, sondern hauptsächlich um die Usability. In der Arbeit möchte ich versuchen unterschiedliche Graphische MockUps zu erstellen und miteinander zu vergleichen. Dazu möchte ich Kriterien aufstellen, wie ich diese Unterschiedlichen Ansätze miteinander vergleichen kann.

Zusätzlich zu dem Graphischen Aufbau der Erweiterung, möchte ich die verschiedenen Optionen evaluieren, wie man am besten die Struktur des Repositories kommt. Dazu gibt es zum einem das Core Schema Notation (CSN), welches die Statische Struktur enthält, sowie einen Language Server, welcher definiert, in welchen Dateien Methoden, Services usw. definiert sind. Die Frage, welche ich mir in meiner Arbeit stellen möchte ist es, dort das optimale Modell zu erstellen, wie man am besten diese beiden Informationsquellen miteinander kombinieren kann. Dabei sollen unterschiedliche Optionen anhand von Kriterien, welche ich vorher definieren möchte abgewägt werden.

1.3 Methodik

Für diese Arbeit wird der Design-Science-Research-Ansatz verwendet, da ein konkretes Artefakt in Form einer VS-Code Extension entwickelt wird.

Zunächst wird das zugrundeliegende Problem analysiert und Anforderungen an die Erweiterung definiert. Anschließend werden verschiedene Ansätze zur Strukturgewinnung aus dem Core Schema Notation (CSN) sowie dem Language Server untersucht und miteinander verglichen. Auf dieser Basis wird ein geeignetes Modell zur Kombination beider Informationsquellen konzipiert.

Parallel dazu werden unterschiedliche grafische Mockups erstellt und anhand definierter Kriterien bewertet. Das ausgewählte Konzept wird prototypisch als VS-Code Extension implementiert und abschließend anhand der zuvor festgelegten Kriterien evaluiert.

2 Zeitplanung

2.1 Plan A

Bei Plan A versuche ich herauszufinden, welche Architektur am besten ist, um die Informationen von CSN und dem Language Server zusammenzuführen. Dort ist der Fokus auf CSN und dem Language Server. Wenn ich hier in Probleme fahre, muss ich Plan B verwenden.

Phase 1: Analyse und Exploration (Woche 1-3)

- CSN und den Language Server analysieren
- erste grobe Implementierungen
- Abwegen, wie realistisch Plan A ist (basierend auf ersten Implementierungen)

Ergebnis: Verständnis; Erster grober Prototyp; Einschätzen wie realistisch Plan A ist

Phase 2: Vergleich unterschiedlicher Ansätze (Woche 4-6)

- Entwicklung unterschiedlicher Ansätze
- Kriterien definieren (Performance)
- mindestens 3 unterschiedliche Modelle (damit die Evaluation sinnvoll wird)

Ergebnis: 3 Modelle für das Problem; Basis für die Evaluation; Wenn es bis hier klappt, ist Plan A realistisch

Phase 3: Umsetzung des Besten Modells (Woche 7-10)

- Implementierung des besten Modells

Ergebnis: fundamentale Entwicklung ist abgeschlossen; ab hier „könnte“ man sich „nur“ noch auf das Schreiben der Arbeit beschränken

Phase 4: Evaluation und Schreiben (Woche 10-12)

- Testen des besten Modells
- Bewertung des Modells anhand der Kriterien

- finale Evaluation

Ergebnis: fertige Arbeit; am besten schon in Woche 11 fertig werden, um Puffer zu haben

2.2 Plan B

Falls Plan A nicht erfolgreich ist, brauche ich noch einen Plan B. Wichtig ist, dass ich relativ früh weiß, ob ich mich auf Plan A verlassen kann, oder ob ich auf Plan B zurückgreifen muss. Ich sollte nach Phase 2: Vergleich unterschiedlicher Ansätze (Woche 4-6) spätestens herausgefunden haben, ob Plan A realistisch ist. Wenn nicht sollte ich auf folgenden Plan B übergehen:

Phase 3: UI- und Strukturkonzepte (Woche 6-8)

- mindestens 3 MockUps entwickeln
- Kriterien definieren für die Mockups

Ergebnis: 3 MockUps für die Evaluation, Kriterien für die Evaluation

Phase 4: Implementierung des besten UI-Konzepts (Woche 8-10)

- Umsetzung des besten MockUps
- Filter und Sortiermöglichkeiten
- mögliche Erweiterbarkeit

Ergebnis: Implementiertes MockUp

Phase 5: Evaluation und schreiben (Woche 10-12)

- Vergleich der UI-Varianten
- Bewertung und Evaluation

Ergebnis: fertige Arbeit; am besten schon in Woche 11 fertig werden, um Puffer zu haben

3 Vorläufige Gliederung

1. **Einleitung**

- 1.1. Motivation
- 1.2. Problemstellung
- 1.3. Zielsetzung der Arbeit
- 1.4. Forschungsfragen
- 1.5. Aufbau der Arbeit

2. **Theoretische Grundlagen**

- 2.1. SAP Cloud Application Programming Model (CAP)
 - 2.1.1. CDS und CDL
 - 2.1.2. Core Schema Notation (CSN)
- 2.2. Language Server und Language Server Protocol
 - 2.2.1. Architektur des Language Servers
 - 2.2.2. Relevante LSP-Funktionalitäten
- 2.3. VS Code Extension Architektur
 - 2.3.1. Tree View API
 - 2.3.2. Integration von Language Servern

3. **Forschungsmethodik**

- 3.1. Design Science Research
- 3.2. Einordnung der Arbeit in den DSR-Prozess
 - 3.2.1. Problemidentifikation
 - 3.2.2. Zieldefinition
 - 3.2.3. Artefaktentwurf
 - 3.2.4. Demonstration
 - 3.2.5. Evaluation

4. **Problem- und Anforderungsanalyse**

- 4.1. Analyse der Repository-Struktur in CAP-Projekten
- 4.2. Funktionale Anforderungen
- 4.3. Nicht-funktionale Anforderungen
- 4.4. Abgrenzung der Arbeit

5. **Designraum der Integrationsstrategien**

- 5.1. Designansatz 1
- 5.2. Designansatz 2

- 5.3. Designansatz 3
- 5.4. Vergleichskriterien für Integrationsansätze
- 6. **Implementierung des Prototyps**
 - 6.1. Technische Architektur der Extension
 - 6.2. Umsetzung der Integrationsansätze
 - 6.3. Visualisierung im VS Code Explorer
 - 6.4. Update- und Synchronisationsstrategie
- 7. **Evaluation**
 - 7.1. Evaluationskriterien
 - 7.2. Vergleich der Integrationsansätze
 - 7.3. Analyse der Ergebnisse
- 8. **Diskussion**
 - 8.1. Interpretation der Ergebnisse
 - 8.2. Limitationen
- 9. **Fazit und Ausblick**
 - 9.1. Zusammenfassung der Ergebnisse
 - 9.2. Beantwortung der Forschungsfragen
 - 9.3. Ausblick auf zukünftige Arbeiten

4 Literaturübersicht

Design Science Research (DSR)

- Design Science Research, A Method for Science and Technology Advancement (ISBN 978-3-319-07373-6)
- Towards an Integrative View on Design Science Research Genres, Strategies, and Pivotal Concepts in Information Systems Research (DOI 10.1145/3571823.3571826)

CAP (CDS, CDL, CSN)

- <https://cap.cloud.sap/docs/cds>

Language Server Protocol (LSP)

- <https://code.visualstudio.com/api/language-extensions/language-server-extension-guide>
- nicht VS-Code Dokumentation (zu technisch, reine Implementierung)

Weitere Literatur

- Clean Code: A Handbook of Agile Software Craftsmanship, 2nd Edition (Robert C. Martin)
- Usability von Software / UX von Software